

1.4 Методы компиляции в WebAssembly

WebAssembly как целевая платформа компиляции поддерживается широким набором инструментариев, различающихся по исходным языкам, архитектуре компилятора и характеристикам генерируемого кода. Выбор конкретного стека определяет архитектуру бинарного модуля, объём сопутствующего рантайма и методику взаимодействия с хост-средой. Рассмотрим основные методы компиляции.

АОТ-компиляция (ahead-of-time). Данный метод предполагает предварительную трансляцию исходного кода непосредственно в бинарный модуль WebAssembly ещё до запуска программы. Именно АОТ-компиляция стала основным способом доставки производительных Wasm-модулей в браузер и серверные рантаймы, поскольку она обеспечивает предсказуемое время инициализации, компактный формат поставки и высокий уровень оптимизации. К инструментам этого класса относятся Emscripten, LLVM/Clang, WASI SDK, Rust с `wasm-bindgen` и `wasm-pack`, TinyGo, AssemblyScript и другие компиляторные цепочки, генерирующие Wasm напрямую.

Плюсы	Минусы
Высокая производительность исполнения	Требуется отдельный этап сборки
Глубокие оптимизации на этапе компиляции	Плохо подходит для динамической генерации кода
Хороший контроль над экспортируемым интерфейсом	Может требовать сложной настройки памяти и рантайма
Удобная интеграция в стандартные сборочные пайплайны	Работа с внешними зависимостями бывает нетривиальной

Таблица 1: Преимущества и недостатки АОТ-компиляции

JIT-компиляция (just-in-time). При данном методе код компилируется во время исполнения, когда уже известен контекст запуска, профиль нагрузки и характеристики среды. В классическом виде JIT чаще применяется не как основной способ получения Wasm-модуля из исходного текста, а как механизм динамической компиляции промежуточного представления или байткода внутри среды выполнения. В экосистеме WebAssembly этот путь встречается заметно реже, чем АОТ, однако важен для рантаймов, где требуется адаптация к данным профилирования и возможность поздней оптимизации. Характерные инструменты и движки этого класса — Wasmer JIT, Cranelift в JIT-сценариях, а также внутренние JIT-пайплайны движков V8 и SpiderMonkey.

Плюсы	Минусы
Оптимизация с учётом реального профиля выполнения	Высокая сложность реализации
Возможность улучшать горячие участки после запуска	Увеличивает время холодного старта
Гибкая адаптация к среде исполнения	Требуется более сложный рантайм
Подходит для сценариев поздней оптимизации	Редко используется как основной метод доставки Wasm

Таблица 2: Преимущества и недостатки JIT-компиляции

Binary translation. Этот метод основан на переводе уже существующего бинарного кода, байткода или инструкций другой архитектуры в представление, пригод-

ное для исполнения в среде WebAssembly. Такой подход применяется тогда, когда исходный код недоступен либо перенос системы с уровня исходников экономически нецелесообразен. В практических и исследовательских решениях сюда можно отнести CheerpX, некоторые QEMU/LLVM-ориентированные схемы динамической трансляции и другие специализированные системы двоичного переноса.

Плюсы	Минусы
Позволяет использовать уже существующий бинарный код	Очень высокая сложность реализации
Не всегда требует доступа к исходникам	Производительность обычно ниже АОТ-компиляции
Полезен для переноса legacy-систем	Результат зависит от качества сопоставления архитектур
Может сокращать затраты на переписывание системы	Используется в основном в нишевых сценариях

Таблица 3: Преимущества и недостатки binary translation

Интерпретация. В интерпретационном подходе в WebAssembly переносится не сама программа в виде нативно скомпилированного модуля, а интерпретатор, который затем исполняет внешний бинарный код, байткод или сценарный язык инструкция за инструкцией. Этот метод особенно удобен для обеспечения совместимости и быстрого запуска существующих экосистем без радикальной переработки их внутренней модели исполнения. К подобным решениям относятся Pyodide с интерпретатором CPython, Lua- и QuickJS-рантаймы, а также различные эмуляторы и интерпретаторы, скомпилированные в Wasm.

Плюсы	Минусы
Высокая гибкость	Производительность ниже АОТ-решений
Сравнительно быстрый перенос сложных экосистем	Часто уступает и JIT-подходам
Удобна для задач совместимости	Каждая инструкция проходит дополнительный уровень обработки
Позволяет сохранить полноту поддержки языка	Плохо подходит для тяжёлых вычислительных задач

Таблица 4: Преимущества и недостатки интерпретации

Hybrid-подход. Гибридная схема объединяет несколько методов компиляции и исполнения в рамках одной системы. На практике это означает, что часть кода может исполняться через интерпретацию, часть — быть заранее скомпилированной АОТ, а наиболее критические участки — дополнительно оптимизироваться специальным рантаймом. Подобные решения характерны прежде всего для управляемых платформ и больших фреймворков, где необходимо одновременно сохранить совместимость, приемлемую скорость запуска и достаточную производительность. Типичными примерами являются Blazor WebAssembly с АОТ-режимом, смешанные пайплайны Mono и ряд managed runtime-решений для Java и .NET.

VM → Wasm. При данном методе в WebAssembly компилируется виртуальная машина или рантайм, а уже внутри него исполняется байткод исходной платформы. В отличие от более общей интерпретации, здесь обычно речь идёт о полноценных управляемых экосистемах с собственным форматом байткода, системой типов, механизмами

Плюсы	Минусы
Гибкий баланс между скоростью запуска и производительностью	Архитектурно более сложен
Позволяет сочетать сильные стороны разных схем исполнения	Требует большего объёма рантайма
Удобен для крупных управляемых платформ	Сложнее в отладке и сопровождении
Помогает сохранять совместимость платформы	Итоговое поведение зависит сразу от нескольких механизмов

Таблица 5: Преимущества и недостатки hybrid-подхода

загрузки классов или сборки мусора. Этот путь особенно распространён для Java- и .NET-платформ, где требуется сохранить существующий стек библиотек и модель исполнения. Наиболее показательные примеры — Blazor WebAssembly с рантаймом Mono, Java-ориентированные решения вроде CheerpJ и другие системы, в которых виртуальная машина переносится в браузер как Wasm-модуль.

Плюсы	Минусы
Высокая совместимость с исходной экосистемой	Увеличивает размер поставляемого модуля
Можно повторно использовать существующий байткод	Даёт более медленный холодный старт
Сохраняет привычную модель исполнения платформы	Создаёт дополнительные накладные расходы рантайма
Облегчает перенос существующих библиотек	Эффективность зависит от качества реализации виртуальной машины

Таблица 6: Преимущества и недостатки метода VM → Wasm

Сопоставление перечисленных методов показывает, что между ними существует устойчивый компромисс. Чем ближе метод к прямой предварительной компиляции, тем выше, как правило, производительность и тем меньше объём вспомогательного рантайма. Чем сильнее система опирается на интерпретацию, виртуальную машину или гибридную модель, тем проще сохранить совместимость с исходной платформой, но тем выше накладные расходы на запуск и исполнение. Сравнительные характеристики основных методов приведены в таблице 7.

Метод	Вход	Производительность	Сложность	Использование
АОТ	Исходный код	Высокая	Средняя	Основной способ сборки Wasm-модулей
JIT	Код во время выполнения, IR, байткод	Средняя	Высокая	Редко, в основном во внутренних рантаймах
Binary translation	Готовый бинарь, машинный код, байткод	Средняя или низкая	Очень высокая	Нишевые сценарии переноса legacy-систем
Интерпретация	Бинарь, байткод, сценарный код	Низкая	Низкая или средняя	Совместимость и быстрый перенос экосистем
Hybrid	Смешанный вход	Средняя	Высокая	Managed runtime и крупные платформы
VM → Wasm	Байткод виртуальной машины	Средняя	Средняя	Java-, .NET- и другие байткодные экосистемы

Таблица 7: Сравнение методов компиляции в WebAssembly